Tutorial 9: Problem Solving- IV: Pipelined Processors

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

Problem 1: Pipelining Performance and Overheads

Assume that an unpipelined processor has a 1ns clock cycle and that it uses 4 cycles for ALU operations and branches and 5 cycles for memory operations. Assume that the relative frequencies of these operations are 40%, 20%, and 40%, respectively. Suppose that due to clock skew and setup, pipelining the processor adds 0.2ns of overhead to the clock. Ignoring any latency impact, how much speedup in the instruction execution rate will we gain from a pipeline?

Solution 1:

This exercise explores the trade-off between increasing instruction throughput via pipelining and the physical constraints imposed by clock skew and register setup times.

Step 1: Analyzing Unpipelined Performance

The baseline performance of the unpipelined processor is determined by the Weighted Average CPI based on instruction frequencies.

- **ALU/Branch:** 4 cycles (60% frequency)
- **Memory:** 5 cycles (40% frequency)
- **Clock Cycle:** 1 ns

Using the formula:

$$\begin{aligned}\text{Average instruction Time}_{\text{unpipelined}} &= \text{Clock} \times \text{Average CPI} \\ &= 1 \text{ ns} \times [(0.4 + 0.2) \times 4 + 0.4 \times 5] \\ &= 1 \text{ ns} \times [2.4 + 2.0] \\ &= 4.4 \text{ ns}\end{aligned}$$

Step 2: Pipelined Clock Cycle and Overhead

In a pipelined version, the clock must accommodate the slowest stage plus the **Pipeline Overhead** (latch delay and clock skew).

$$\text{Average instruction Time}_{\text{pipelined}} = \text{Logic Time} + \text{Overhead} = 1 \text{ ns} + 0.2 \text{ ns} = 1.2 \text{ ns}$$

Even with an ideal CPI of 1, each instruction now retires at a rate of 1.2 ns per clock cycle.

Step 3: Calculating Speedup

The speedup is the ratio of the unpipelined execution time to the pipelined execution time:

$$\text{Speedup} = \frac{\text{Average instruction Time}_{\text{unpipelined}}}{\text{Average instruction Time}_{\text{pipelined}}} = \frac{4.4 \text{ ns}}{1.2 \text{ ns}} \approx 3.67$$

Step 4: The Diminishing Returns (Amdahl's Law)

The 0.2 ns overhead essentially establishes a limit on the effectiveness of pipelining.

- **The Bottleneck:** As the number of stages $n \to \infty$, the logic time per stage approaches 0 ns.
- **The Ultimate Limit:** The clock cycle time can never be less than the overhead (0.2 ns).
- **Theoretical Maximum:** No matter how deep the pipeline becomes, the maximum speedup is capped at $\frac{4.4 \text{ ns}}{0.2 \text{ ns}} = 22\times$.

Once the clock cycle is as small as the sum of the clock skew and latch overhead, no further pipelining is useful, as there is no time left in the cycle for actual logic work.

Problem 2: Pipelining Performance and Speedup

Consider a machine with a single-cycle implementation and a clock cycle time of 7 ns. The stages are split into five functional units with the following delays: IF = 1 ns; ID = 1.5 ns; EX = 1 ns; MEM = 2 ns; and WB = 1.5 ns. The pipeline register delay is 0.1 ns.

a. What is the clock cycle time of the 5-stage pipelined machine?

b. If there is a stall every 4 instructions, what is the CPI of the new machine?

c. What is the speedup of the pipelined machine over the single-cycle machine?

d. If the pipelined machine had an infinite number of stages, what would its speedup be over the single-cycle machine?

Solution 2:

a. Clock Cycle Time

In a pipelined processor, the clock cycle time (T_{cycle}) is determined by the delay of the slowest stage plus the overhead of the pipeline register delay.

$$\begin{aligned} T_{cycle} &= \max(\text{IF, ID, EX, MEM, WB}) + \text{Register Delay} \\ T_{cycle} &= 2.0\,\text{ns} + 0.1\,\text{ns} \\ T_{cycle} &= \mathbf{2.1}\,\text{ns} \end{aligned}$$

b. CPI Calculation

The ideal CPI for a pipeline is 1.0. If a stall occurs every 4 instructions, we add the stall penalty (1 cycle) averaged over those instructions:

$$\begin{aligned} \text{CPI}_{new} &= \text{Ideal CPI} + \frac{\text{Stall Cycles}}{\text{Number of Instructions}} \\ \text{CPI}_{new} &= 1.0 + \frac{1}{4} \\ \text{CPI}_{new} &= \mathbf{1.25} \end{aligned}$$

c. Speedup Calculation

Speedup is calculated as the ratio of the execution time of the single-cycle machine to the execution time of the pipelined machine:

Without stall:

$$\begin{aligned} \text{Speedup} &= \frac{\text{CPI}_{old} \times \text{Clock}_{old}}{\text{CPI}_{new} \times \text{Clock}_{new}} \\ \text{Speedup} &= \frac{1 \times 7\,\text{ns}}{1.0 \times 2.1\,\text{ns}} \\ \text{Speedup} &= \frac{7}{2.1} \approx \mathbf{3.33} \end{aligned}$$

With stall:

$$\begin{aligned} \text{Speedup} &= \frac{\text{CPI}_{old} \times \text{Clock}_{old}}{\text{CPI}_{new} \times \text{Clock}_{new}} \\ \text{Speedup} &= \frac{1 \times 7\,\text{ns}}{1.25 \times 2.1\,\text{ns}} \\ \text{Speedup} &= \frac{7}{2.625} \approx \mathbf{2.67} \end{aligned}$$

d. Infinite Stages Speedup

As the number of stages approaches infinity, the logic delay per stage approaches 0 ns. However, each stage still requires a pipeline register. Therefore, the clock cycle time becomes equal to the register delay (0.1 ns). Assuming an ideal CPI of 1 for the infinite pipeline:

$$\begin{aligned} \text{Clock}_{\infty} &= 0.1\,\text{ns} \\ \text{Speedup}_{\infty} &= \frac{1 \times 7\,\text{ns}}{1 \times 0.1\,\text{ns}} \\ \text{Speedup}_{\infty} &= \mathbf{70} \end{aligned}$$

Note: This theoretical maximum illustrates that the pipeline register overhead is the ultimate bottleneck for performance scaling.

Problem 3: Pipeline Register Overhead and Throughput

A single-cycle processor takes 120 ns per instruction. It is perfectly pipelined into 12 stages with pipeline registers that each introduce a 2.5 ns propagation delay.

a. Calculate the clock cycle time of the pipelined processor.

b. Determine the ideal speedup of the pipelined processor over the single-cycle processor, assuming a continuous stream of instructions with no hazards.

Solution 3: Pipeline Register Overhead and Throughput

Part a: Pipelined Clock Cycle Time

The single-cycle instruction takes 120 ns. When perfectly divided into 12 equal pipeline stages, the logic delay per stage is:

$$\text{Logic Delay per Stage} = \frac{120\,\text{ns}}{12} = 10\,\text{ns}$$

However, each pipeline stage must account for the propagation delay introduced by the pipeline registers.

$$\begin{aligned} T_{cycle\_pipe} &= \text{Logic Delay per Stage} + \text{Register Delay} \\ T_{cycle\_pipe} &= 10\,\text{ns} + 2.5\,\text{ns} = \mathbf{12.5}\,\text{ns} \end{aligned}$$

Part b: Ideal Speedup

Assuming a continuous stream of instructions with no hazards, the ideal CPI for both processors is 1.

$$\text{Speedup} = \frac{\text{Time}_{SC}}{\text{Time}_{Pipe}} = \frac{T_{cycle\_SC}}{T_{cycle\_pipe}}$$

$$\text{Speedup} = \frac{120\,\text{ns}}{12.5\,\text{ns}} = \mathbf{9.6}$$

Problem 4: Splitting Pipeline Stages

Consider a 5-stage pipeline with latencies of 200 ps, 150 ps, 400 ps, 250 ps, and 100 ps.

a. If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage (assuming no additional overhead), which stage would you split to achieve the maximum reduction in clock cycle time?

b. What is the new clock cycle time, and what is the resulting ideal speedup over the original 5-stage pipeline?

Solution 4: Splitting Pipeline Stages

Part a: Identifying the Bottleneck

In a pipelined processor, the clock cycle time is dictated by the slowest stage (the bottleneck). The latencies of the 5 stages are: 200 ps, 150 ps, **400 ps**, 250 ps, and 100 ps. The original clock cycle time is $\max(200, 150, 400, 250, 100) = \mathbf{400}\,\text{ps}$.
To achieve the maximum reduction in clock cycle time, we must split the longest stage. Therefore, we should split the **400 ps stage**.

Part b: New Clock Cycle and Speedup

If we split the 400 ps stage into two perfectly equal stages with no overhead, the new pipeline will have 6 stages with the following latencies: 200 ps, 150 ps, **200 ps, 200 ps**, 250 ps, and 100 ps.

- **New Clock Cycle Time:** The new bottleneck is the 250 ps stage.

$$T_{cycle\_new} = \max(200, 150, 200, 200, \mathbf{250}, 100) = \mathbf{250}\,\text{ps}$$

- **Ideal Speedup:** Assuming an ideal CPI of 1 for both pipelines, the speedup is the ratio of their clock cycle times.

$$\text{Speedup} = \frac{T_{cycle\_old}}{T_{cycle\_new}} = \frac{400\,\text{ps}}{250\,\text{ps}} = \mathbf{1.6}$$

Problem 5: Control Stalls and Branch Prediction

A pipelined processor operating at 2.5 GHz has a standard 5-stage instruction pipeline. Under ideal conditions, it has a base CPI of 1. However, for a specific program, 25% of the instructions are branches. Without any advanced hardware, every branch instruction incurs a flat 2-cycle stall due to control resolution delays.
A designer proposes a new version of the processor that includes a Branch Predictor Unit (BPU). The BPU eliminates stalls entirely for correctly predicted branches, but incurs a 3-cycle stall penalty for every wrong prediction.

a. Calculate the average CPI and total execution time for 10,000 instructions on the original processor.

b. If the new BPU has a prediction accuracy of 80%, calculate the new average CPI.

c. What is the overall speedup achieved by the new processor version over the original one?

Solution 5: Control Stalls and Branch Prediction

First, let us determine the clock cycle time for the 2.5 GHz processor:

$$T_{cycle} = \frac{1}{2.5 \times 10^9\,\text{Hz}} = 0.4\,\text{ns}$$

Part a: Original Processor Performance

The base CPI is 1. Branches constitute 25% of instructions, and each incurs a 2-cycle stall.

$$\begin{aligned}\text{Average CPI}_{old} &= \text{Base CPI} + (\text{Branch Frequency} \times \text{Stall Penalty}) \\ &= 1.0 + (0.25 \times 2) = \mathbf{1.5}\end{aligned}$$

The total execution time for 10,000 instructions:

$$\begin{aligned}\text{Execution Time}_{old} &= \text{Instruction Count} \times \text{CPI}_{old} \times T_{cycle} \\ &= 10,000 \times 1.5 \times 0.4\,\text{ns} = 6,000\,\text{ns} = \mathbf{6}\,\mu\text{s}\end{aligned}$$

Part b: New Processor with BPU

The Branch Predictor Unit (BPU) has an 80% accuracy rate.

- Correctly predicted branches (80% of 25%): 0 stall penalty.
- Mispredicted branches (20% of 25%): 3 stall penalty.

$$\begin{aligned}\text{Average CPI}_{new} &= \text{Base CPI} + \text{Branch Freq} \times [(\text{Correct Rate} \times 0) + (\text{Miss Rate} \times 3)] \\ &= 1.0 + 0.25 \times [(0.80 \times 0) + (0.20 \times 3)] \\ &= 1.0 + 0.25 \times 0.6 \\ &= 1.0 + 0.15 = \mathbf{1.15}\end{aligned}$$

Part c: Speedup

Since the clock frequency is identical for both versions, the speedup is simply the ratio of their average CPIs.

$$\text{Speedup} = \frac{\text{CPI}_{old}}{\text{CPI}_{new}} = \frac{1.5}{1.15} \approx \mathbf{1.30}$$

The new processor is approximately 1.30 times faster.

Problem 6: Performance Comparison of Pipelined vs. SC vs. MC

We are evaluating three different CPU implementations for a given program: a Single-Cycle (SC) processor, a Multi-Cycle (MC) processor, and a Pipelined processor.
The program consists of 200 instructions with the following breakdown: 50% Arithmetic (4 cycles in MC), 20% Load (5 cycles in MC), 15% Store (4 cycles in MC), and 15% Branch (3 cycles in MC).

- The SC processor has a clock frequency of 50 MHz.
- The MC processor divides the clock cycle into shorter steps, running at 250 MHz.
- The Pipelined processor runs at 200 MHz but, due to minor pipeline overheads and occasional stalls, operates with an average CPI of 1.2 instead of the ideal 1.

a. Calculate the total execution time (in microseconds) for the program on the Single-Cycle and Multi-Cycle implementations.

b. Calculate the total execution time for the program on the Pipelined processor.

c. Evaluate and compare the speedup of the Pipelined processor relative to *both* the SC and MC processors.

Solution 6: Performance Comparison of Pipelined vs. SC vs. MC

Part a: Execution Time for SC and MC

Single-Cycle (SC) Processor:

- Clock Rate: 50 MHz $\implies T_{cycle} = 20\,\text{ns}$
- CPI: 1 (by definition)

$$\text{Time}_{SC} = 200 \times 1 \times 20\,\text{ns} = 4,000\,\text{ns} = \mathbf{4.0}\,\mu\text{s}$$

Multi-Cycle (MC) Processor:

- Clock Rate: 250 MHz $\implies T_{cycle} = 4\,\text{ns}$
- Average CPI $= (0.50\times4)+(0.20\times5)+(0.15\times4)+(0.15\times3) = 2.0+1.0+0.6+0.45 =$ **4.05**

$$\text{Time}_{MC} = 200 \times 4.05 \times 4\,\text{ns} = 3,240\,\text{ns} = \mathbf{3.24}\,\mu\text{s}$$

Part b: Execution Time for Pipelined Processor

Pipelined Processor:

- Clock Rate: 200 MHz $\implies T_{cycle} = 5\,\text{ns}$
- Average CPI = 1.2

$$\text{Time}_{Pipe} = 200 \times 1.2 \times 5\,\text{ns} = 1,200\,\text{ns} = \mathbf{1.2}\,\mu\text{s}$$

Part c: Speedup Evaluation

The speedup of the pipelined processor is evaluated against both previous designs:

$$\text{Speedup over SC} = \frac{\text{Time}_{SC}}{\text{Time}_{Pipe}} = \frac{4.0\,\mu\text{s}}{1.2\,\mu\text{s}} \approx \mathbf{3.33}$$

$$\text{Speedup over MC} = \frac{\text{Time}_{MC}}{\text{Time}_{Pipe}} = \frac{3.24\,\mu\text{s}}{1.2\,\mu\text{s}} = \mathbf{2.70}$$

The pipelined processor is significantly faster than both the SC and MC implementations.

Problem 7: Pipelining Instruction Dependencies

Consider a sequence of 4 MIPS instructions executing on a classic 5-stage pipeline (Instruction Fetch, Instruction Decode, Execute, Memory Access, Write Back).

```
I1: add $t1, $t2, $t3
I2: sub $t4, $t1, $t5
I3: and $t6, $t4, $t7
I4: or  $t8, $t2, $t3
```

Notice that `I2` requires the value of `$t1` computed by `I1`, and `I3` requires the value of `$t4` computed by `I2`.
Recall how the pipeline stages interact with the processor's registers: an instruction **reads** its source registers during the Instruction Decode (**ID**) stage, and **writes** its computed result to the destination register only at the end of the Write Back (**WB**) stage. Assume no special hardware exists to bypass this rule; a value must be fully written to the register file by one instruction before a subsequent instruction can read it.

a. Draw a pipeline diagram for correct program execution. (Hint: Carefully look at the specific clock cycles where `I2` reads and `I1` writes. Think about what should you do for correctness).

b. Count the total number of clock cycles required to execute these 4 instructions.

Solution 7: Pipelining Instruction Dependencies

Part a: Pipeline Diagram and Dependency Analysis

Logical Reasoning for Correctness:

- **Data Hazard 1 (`I1` to `I2`):** `I1` computes the value of `$t1`. Because there is no special hardware for bypassing/forwarding, `I1` must fully write `$t1` to the register file at the **end of its Write Back (WB) stage**.
- `I1` is in its WB stage during **Cycle 5**. Therefore, the earliest `I2` can successfully read the correct value of `$t1` in its Instruction Decode (ID) stage is **Cycle 6**.
- To achieve this, the hardware must insert **3 stalls or NOPs*** between `I1` and `I2`. Each stall acts as a dummy instruction flowing through all 5 stages. *Note that NOPs should be implemented carefully and should not interfere with other instruction logic; a safe NOP is:

```
add $0, $0, $0
```

- **Data Hazard 2 (`I2` to `I3`):** `I2` computes the value of `$t4`. Because `I2` was delayed, its WB stage does not occur until **Cycle 9**.
- Therefore, the earliest `I3` can read `$t4` in its ID stage is **Cycle 10**. This requires inserting another **3 stalls** between `I2` and `I3`.
- **`I4`:** This instruction reads `$t2` and `$t3`, which are not modified by any active instruction. It has no dependencies, so it can be fetched immediately after `I3` without any additional stalls.

Table 1: Pipeline Execution with Stalls (ST)

	Clock Cycle Number													
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**
`I1: add $t1, ...`	IF	ID	EX	MEM	WB									
`stall (NOP)`		ST	ST	ST	ST	ST								
`stall (NOP)`			ST	ST	ST	ST	ST							
`stall (NOP)`				ST	ST	ST	ST	ST						
`I2: sub $t4, ...`					IF	ID	EX	MEM	WB					
`stall (NOP)`						ST	ST	ST	ST	ST				
`stall (NOP)`							ST	ST	ST	ST	ST			
`stall (NOP)`								ST	ST	ST	ST	ST		
`I3: and $t6, ...`									IF	ID	EX	MEM	WB	
`I4: or $t8, ...`										IF	ID	EX	MEM	WB

Part b: Total Clock Cycles

By tracing the execution in the pipeline diagram:

- `I1` begins fetching in **Cycle 1**.
- `I4` completes its Write Back (WB) stage at the end of **Cycle 14**.

The total number of clock cycles required to correctly execute these 4 instructions is **14 cycles**.

Problem 8: Cost of a Structural Hazard

Suppose that data references (assume single memory common for data and instructions) constitute 40% of the instruction mix, and the ideal CPI of a pipelined processor (ignoring structural hazards) is 1. Assume a processor with a structural hazard (resulting in stalls) has a clock rate 1.05 times higher than a processor that handles the hazard to avoid stalls. Disregarding any other performance losses, is the pipeline with or without the stalls faster, and by how much?

Table 2: A pipeline stalled for a structural hazard—a load with one memory port. Note the assumption that instructions i + 1 and i + 2 are not memory references.

	Clock cycle number									
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i+1$		IF	ID	EX	MEM	WB				
Instruction $i+2$			IF	ID	EX	MEM	WB			
Instruction $i+3$				**Stall**	IF	ID	EX	MEM	WB	
Instruction $i+4$						IF	ID	EX	MEM	WB
Instruction $i+5$							IF	ID	EX	MEM
Instruction $i+6$								IF	ID	EX

Solution 8:

The structural hazard occurs in a system with a single memory port because the **Instruction Fetch (IF)** and **Data Access (MEM)** stages cannot occur simultaneously. This conflict results in a pipeline stall for every data reference.

Table 3: Pipeline Timing Diagram showing Structural Hazard

Instruction	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**C7**	**C8**
Load Instruction	IF	ID	EX	**MEM**	WB			
Instruction $i+1$		IF	ID	EX	MEM	WB		
Instruction $i+2$			IF	ID	EX	MEM	WB	
Instruction $i+3$				**STALL**	IF	ID	EX	MEM

As shown in the timing diagram, the Load instruction in Cycle 4 "steals" the memory port for data access, forcing Instruction $i+3$ to wait. This confirms that each data reference adds exactly **1 stall cycle** to the execution.
The performance of a processor is determined by the *Average Instruction Time*:

$$\text{Avg. Instruction Time} = \text{CPI} \times \text{Clock Cycle Time} \tag{1}$$

Calculating the Average CPI for the hazard-prone processor:

$$\begin{aligned} \text{Avg. CPI} &= \text{BASE CPI} + (BranchFrequency \times StallPenalty) \\ &= (1.0 + 0.4 \times 1) \\ &= 1.4 \end{aligned}$$

Let T_{ideal} be the clock cycle time of the processor without the hazard. The clock cycle

time for the processor with the hazard (T_{hazard}) is:

$$T_{\text{hazard}} = \frac{T_{\text{ideal}}}{1.05} \tag{2}$$

Calculating the Average Instruction Time for the hazard-prone processor:

$$\begin{aligned}
\text{Avg. Time}_{\text{hazard}} &= \text{CPI}_{\text{hazard}} \times T_{\text{hazard}} \\
&= (1.0 + 0.4 \times 1) \times \frac{T_{\text{ideal}}}{1.05} \\
&= \frac{1.4}{1.05} \times T_{\text{ideal}} \\
&\approx 1.33 \times T_{\text{ideal}}
\end{aligned}$$

Conclusion

The processor **without** the structural hazard is faster. By comparing the ratios of the average instruction times:

$$\frac{\text{Avg. Time}_{\text{hazard}}}{\text{Avg. Time}_{\text{ideal}}} = \frac{1.33 \times T_{\text{ideal}}}{1 \times T_{\text{ideal}}} = 1.33 \tag{3}$$

Even with a 5% faster clock, the increased CPI makes the hazard-prone processor **1.33 times slower** than the ideal pipeline.